USTHB
Faculté d'Informatique
Laboratoire MOVEP

TD / TP MFGL
Ing 5 Soft.Eng
Année 2026-2027

MÉTHODES FORMELLES POUR LE GÉNIE LOGICIEL

Réservation de places : spécifier, vérifier, valider

Spécification formelle, modélisation et vérification d'un système de réservation de places pour un événement de capacité limitée, du contrat Java jusqu'au model checking probabiliste.

RÉALISÉ PAR
Bechar Walid
Groupe 2

ENCADRÉ PAR
Dr Mekahlia Fatma Zohra

DATE
Novembre 2026

SYNTHÈSE

En bref

Le système doit garantir qu'une place n'est jamais attribuée deux fois, même lorsque deux utilisateurs demandent la dernière place au même moment. Nous avons spécifié l'opération de réservation, l'avons implémentée avec des contrats vérifiés, puis modélisée avec cinq outils formels. Le résultat central : une réservation **non atomique** permet la double réservation (NuSMV en donne le contre-exemple), alors que la version **atomique** satisfait toutes les propriétés.

10/10
tests Java réussis, contrats respectés

12/12
propriétés CTL/LTL vraies sur le modèle correct

5 états
contre-exemple de double réservation (modèle incorrect)

97,2 %
probabilité de confirmation (PRISM)

Correspondance avec le travail demandé

TRAVAIL DEMANDÉ	LIVRABLE	OUTIL	SECTION
Préconditions, postconditions, invariants	Contrat de `reserver`	Notation formelle	1
Réservation séquentielle en Java, vérification des conditions	`1-java/`	Java	2
États d'une réservation, transitions autorisées	`etats.puml`, `etats.jff`	PlantUML, JFLAP	3
Deux utilisateurs demandent la dernière place	`concurrence.puml`	PlantUML	4
Disponibilité, transitions et conflits	`reservation.pnml`	PIPE	5
Propriétés de sûreté et de vivacité, formalisation temporelle	Formules CTL/LTL	NuSMV	6
Scénario incorrect, vérification, contre-exemples	`4-nusmv/`	NuSMV	7
Probabilités de défaillance ou de retard, confirmation avant échéance	`5-prism/`	PRISM	8

Sommaire

0 Contexte et hypothèses
1 Spécification
2 Implémentation Java
3 États et transitions
4 Scénario concurrent
5 Réseau de Petri
6 Propriétés temporelles
7 Vérification NuSMV
8 Modèle probabiliste
9 Conclusion
A Annexes : sources

SECTION 0

Contexte et hypothèses

Un événement dispose d'une capacité `N` fixée à l'avance. Les utilisateurs soumettent des demandes de réservation : si une place est libre, la demande est confirmée, sinon elle est refusée. Deux contraintes doivent toujours tenir : le nombre de places disponibles reste cohérent, et aucune place n'est attribuée deux fois. La difficulté apparaît en concurrence, quand deux demandes simultanées visent la dernière place.

Hypothèses de modélisation

- **Capacité et utilisateurs.** Les modèles formels (Petri, NuSMV) utilisent `N = 1` et deux utilisateurs : c'est exactement la situation critique de la dernière place. L'implémentation Java est générale (capacité quelconque).
- **Annulation.** Une demande peut être annulée depuis *En attente* (l'utilisateur renonce) ou depuis *Confirmée* (la place est alors libérée). *Refusée* et *Annulée* sont des états finaux.
- **Concurrence.** NuSMV modélise un entrelacement : à chaque pas, un ordonnanceur choisit l'utilisateur qui agit. Les propriétés de vivacité supposent un ordonnanceur équitable.
- **Temps.** Le modèle PRISM est une chaîne de Markov à temps discret : un pas représente une unité de temps.

Outils utilisés

OUTIL	VERSION	USAGE
Java (OpenJDK)	25	Implémentation séquentielle et tests
PlantUML	1.2024.7	Diagramme d'états, diagramme de séquence
JFLAP	7.1	Automate des états, simulation de séquences
PIPE	5.0.2	Réseau de Petri, graphe d'accessibilité
NuSMV	2.7.1	Vérification CTL/LTL, contre-exemples
PRISM	4.11	Vérification probabiliste (PCTL)

SECTION 1

Spécification de l'opération de réservation

L'état du système est décrit par trois variables : la capacité `N`, le nombre de places libres `placesLibres` et le nombre de réservations confirmées `nbConfirmees`. Chaque demande `d` possède un état parmi *EN_ATTENTE*, *CONFIRMEE*, *REFUSEE* et *ANNULEE*.

INVARIANT

$$0 \le placesLibres \le N$$
$$placesLibres + nbConfirmees = N$$

PRÉCONDITIONS DE RESERVER(D)

$$d \neq null \ \wedge \ d.etat = EN_ATTENTE \ \wedge \ Invariant$$

POSTCONDITIONS DE RESERVER(D)

$$placesLibres_0 > 0 \ \Rightarrow \ d.etat = CONFIRMEE \ \wedge \ placesLibres = placesLibres_0 - 1$$
$$placesLibres_0 = 0 \ \Rightarrow \ d.etat = REFUSEE \ \wedge \ placesLibres = placesLibres_0$$
$$Invariant$$

L'indice $_0$ désigne la valeur avant l'appel. La première ligne de l'invariant interdit la sur-réservation (jamais moins de zéro place libre). La seconde garantit qu'aucune place n'est perdue ni comptée deux fois.

PRÉCONDITIONS DE ANNULER(D)

$$d.etat \in \{EN_ATTENTE, CONFIRMEE\} \ \wedge \ Invariant$$

POSTCONDITIONS DE ANNULER(D)

$$d.etat = ANNULEE$$
$$d.etat_0 = CONFIRMEE \ \Rightarrow \ placesLibres = placesLibres_0 + 1$$
$$d.etat_0 = EN_ATTENTE \Rightarrow \ placesLibres = placesLibres_0$$
$$Invariant$$

SECTION 2

Implémentation Java séquentielle

La classe `SystemeReservation` traduit directement le contrat : chaque méthode vérifie ses préconditions et l'invariant à l'entrée, puis ses postconditions et l'invariant à la sortie. Une violation lève une `IllegalStateException`. Les vérifications sont explicites (et non des `assert`) pour rester actives sans option de la JVM.

```
/**
 * Pre  : d != null et d est EN_ATTENTE.
 * Post : si une place etait libre, d est CONFIRMEE et placesLibres a diminue de 1,
 *        sinon d est REFUSEE et placesLibres n'a pas change.
 */
public void reserver(Demande d) {
    // Preconditions
    exiger(d != null, "pre : la demande ne doit pas etre nulle");
    exiger(d.getEtat() == Etat.EN_ATTENTE, "pre : la demande doit etre EN_ATTENTE");
    verifierInvariant();
```

```
        int avant = placesLibres;

        if (placesLibres > 0) {
            placesLibres--;
            nbConfirmees++;
            d.setEtat(Etat.CONFIRMEE);
        } else {
            d.setEtat(Etat.REFUSEE);
        }

        // Postconditions
        if (avant > 0) {
            exiger(d.getEtat() == Etat.CONFIRMEE && placesLibres == avant - 1,
                   "post : une place etait libre, la demande doit etre confirmee");
        } else {
            exiger(d.getEtat() == Etat.REFUSEE && placesLibres == avant,
                   "post : aucune place libre, la demande doit etre refusee");
        }
        verifierInvariant();
    }
```

```
cd 1-java && javac -d out src/*.java && java -cp out Main
```

Résultats des tests (10/10 tests réussis)

Scénario avec une capacité de 2 : A et B sont confirmées, C est refusée, A annule, puis D obtient la place libérée. Les appels qui violent une précondition doivent être rejetés.

TEST	RÉSULTAT
A et B confirmées	**réussi**
C refusée (plus de place)	**réussi**
0 place libre	**réussi**
A annulée, une place libérée	**réussi**
D obtient la place libérée	**réussi**
réserver une demande déjà confirmée rejeté (pré : la demande doit être EN_ATTENTE)	**réussi**
réserver une demande refusée rejeté (pré : la demande doit être EN_ATTENTE)	**réussi**
annuler une demande refusée rejeté (pré : seule une demande en attente ou confirmée peut être annulée)	**réussi**
annuler deux fois rejeté (pré : seule une demande en attente ou confirmée peut être annulée)	**réussi**
capacité nulle rejeté (la capacité doit être positive)	**réussi**

SECTION 3

États et transitions d'une réservation

Une demande naît *En attente*. La réservation la conduit vers *Confirmée* ou *Refusée* selon les places disponibles ; l'annulation la conduit vers *Annulée*. Les gardes et les effets sur `placesLibres` reprennent le contrat de la section 1.

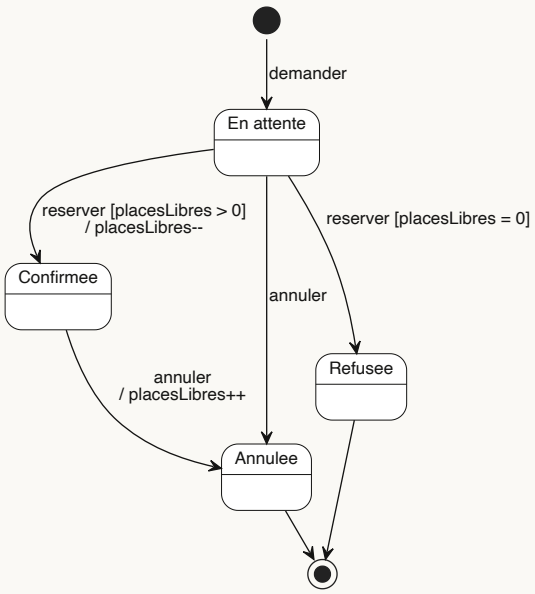


Figure 1. Diagramme d'états (PlantUML, `2-etats/etats.puml`).

Automate JFLAP et simulation

L'automate fini `2-etats/etats.jff` encode les mêmes transitions avec l'alphabet `c` (réserver avec place libre), `r` (réserver sans place libre) et `a` (annuler). Une séquence est acceptée si elle respecte les transitions autorisées. Le fichier a été chargé et simulé avec le moteur de JFLAP 7.1 (équivalent de *Input › Multiple Run*).

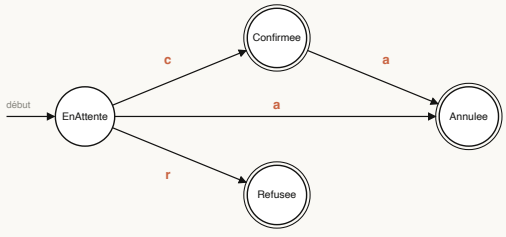


Figure 2. Automate JFLAP. États finaux en double cercle.

SÉQUENCES	RÉSULTAT	SENS
`c` `r` `a`	**acceptées**	confirmation, refus, abandon
`ca`	**acceptée**	confirmer puis annuler
`cc` `rc` `ac`	**rejetées**	double confirmation, confirmer après un refus ou une annulation
`rr` `aa` `cac`	**rejetées**	sortir d'un état final

SECTION 4

Deux utilisateurs, une dernière place

Le diagramme de séquence compare deux exécutions lorsque `placesLibres = 1`. Dans la version incorrecte, la vérification de la place et la décrémentation sont deux étapes séparées : les deux utilisateurs voient la place libre avant que l'un d'eux ne la prenne. Dans la version correcte, les deux opérations forment une seule action atomique protégée par un verrou.

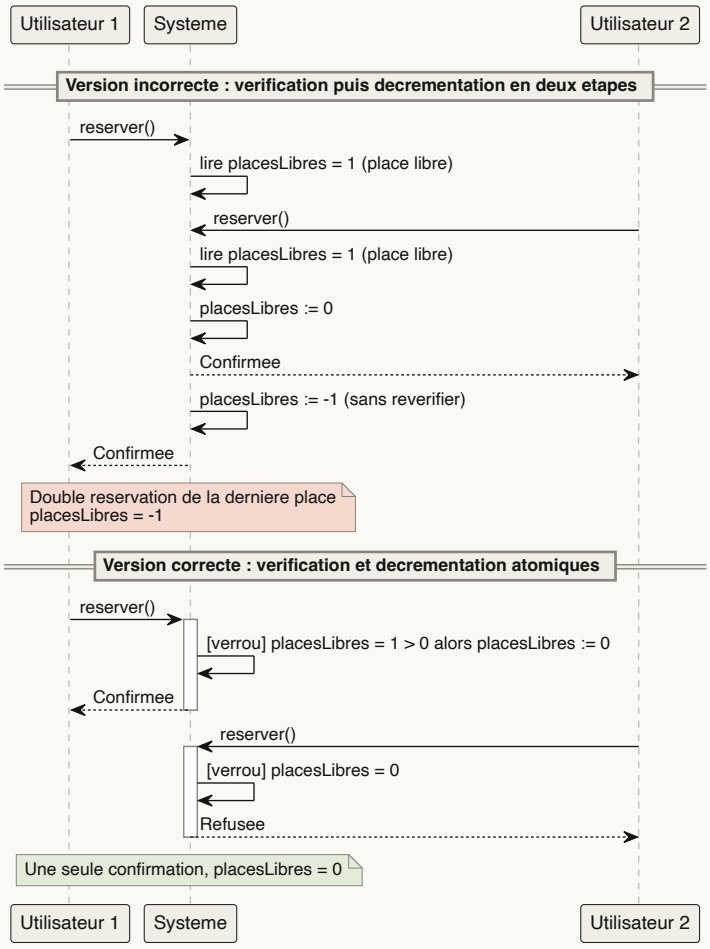


Figure 3. Scénario concurrent (PlantUML, `2-etats/concurrence.puml`). L'ordre des étapes de la version incorrecte reprend le contre-exemple produit par NuSMV (section 7).

SECTION 5

Réseau de Petri : disponibilité et conflits

Le réseau `3-petri/reservation.pnml` (format PIPE) modélise deux utilisateurs et une place. La place `Libres` contient un jeton par place disponible, `Occupees` un jeton par place attribuée. Chaque utilisateur possède un jeton qui circule entre ses quatre états.

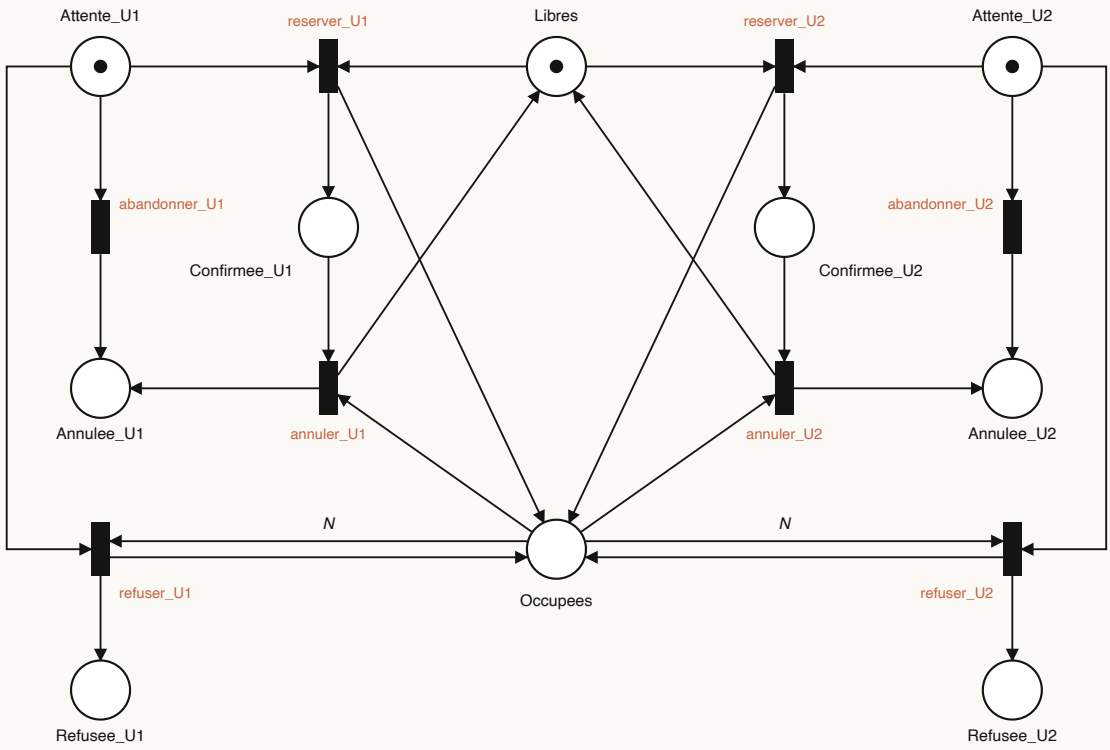


Figure 4. Réseau de Petri, marquage initial (`N = 1`). Les transitions sont en noir, leurs noms en orange.

- `reserver_Ui` consomme le jeton de `Libres` et le place dans `Occupees`.
- `refuser_Ui` n'est franchissable que si `Occupees` contient `N` jetons (arcs de poids `N` dans les deux sens). C'est un test à zéro de `Libres` sans arc inhibiteur.
- `annuler_Ui` rend le jeton à `Libres` ; `abandonner_Ui` annule une demande encore en attente.

> **Conflit sur la dernière place.** Dans le marquage initial, `reserver_U1` et `reserver_U2` sont toutes deux franchissables mais se disputent l'unique jeton de `Libres` : le franchissement de l'une désactive l'autre. Le réseau rend ainsi la réservation atomique par construction.

Analyse du graphe d'accessibilité

Le graphe a été exploré avec le moteur d'animation de PIPE 5.0.2.

PROPRIÉTÉ	RÉSULTAT
Marquages accessibles	12
Transitions franchissables au départ	`reserver_U1`, `reserver_U2`, `abandonner_U1`, `abandonner_U2`
Borné	**oui**, 1-borné (réseau sauf)
Invariant de places `Libres + Occupees = N`	**vérifié** dans les 12 marquages
Double réservation `Confirmee_U1 + Confirmee_U2 > N`	**jamais atteinte**
Blocages	aucun : les 3 marquages morts sont des états finaux (refus ou annulation)

SECTION 6

Propriétés de sûreté et de vivacité

Une propriété de **sûreté** affirme que rien de mauvais n'arrive jamais (forme `AG ¬mauvais`). Une propriété de **vivacité** affirme que quelque chose de bon finit par arriver (forme `AF bon` ou `EF bon`). Les formules portent sur le modèle NuSMV, où `u1` et `u2` sont les états des deux demandes et `libres` le nombre de places disponibles.

	PROPRIÉTÉ	FORMULE
S1	Places libres dans `[0, N]`	`AG (libres >= 0 & libres <= N)`
S2	Cohérence du nombre de places	`AG (libres + nb_confirmees = N)`
S3	Pas de double réservation	`AG !(u1 = confirmee & u2 = confirmee)`
S4	Refus et annulation définitifs	`AG (u1 = refusee -> AG u1 = refusee)`
S5	Refus seulement si complet	`AG ((u1 = attente & libres > 0) -> AX u1 != refusee)`
V1	Toute demande reçoit une réponse	`AG (u1 = attente -> AF u1 != attente)`
V2	Chacun peut obtenir la place	`EF (u1 = confirmee)`
V3	Les deux demandes sont traitées (LTL)	`F (u1 != attente & u2 != attente)`
V4	Une place libérée peut être reprise	`AG ((u1 = annulee & u2 = attente) -> EF u2 = confirmee)`

S4, V1 et V2 sont vérifiées pour les deux utilisateurs. Les propriétés de vivacité supposent un ordonnanceur équitable, déclaré par `FAIRNESS qui = 1` et `FAIRNESS qui = 2` : sans cette hypothèse, un utilisateur pourrait ne jamais être servi.

SECTION 7

Vérification avec NuSMV

Deux modèles partagent les mêmes variables et les mêmes propriétés. Seul le protocole de réservation change :

- `reservation_correcte.smv` : la vérification et la décrémentation se font en un seul pas.
- `reservation_bug.smv` : l'utilisateur passe d'abord par un état interne `verifiee` (il a vu une place libre), puis confirme et décrémente plus tard **sans revérifier**.

```
NuSMV 4-nusmv/reservation_correcte.smv
NuSMV 4-nusmv/reservation_bug.smv
```

	SPÉCIFICATION NUSMV	MODÈLE CORRECT 24 ÉTATS	MODÈLE INCORRECT 40 ÉTATS
S1	`AG (libres >= 0 & libres <= N)`	vraie	fausse
S2	`AG libres + nb_confirmees = N`	vraie	vraie
S3	`AG !(u1 = confirmee & u2 = confirmee)`	vraie	fausse
S4	`AG (u1 = refusee -> AG u1 = refusee)`	vraie	non vérifiée
S4	`AG (u1 = annulee -> AG u1 = annulee)`	vraie	non vérifiée
S5	`AG ((u1 = attente & libres > 0) -> AX u1 != refusee)`	vraie	non vérifiée
V1	`AG (u1 = attente -> AF u1 != attente)`	vraie	vraie
V1	`AG (u2 = attente -> AF u2 != attente)`	vraie	non vérifiée
V2	`EF u1 = confirmee`	vraie	non vérifiée
V2	`EF u2 = confirmee`	vraie	non vérifiée
V4	`AG ((u1 = annulee & u2 = attente) -> EF u2 = confirmee)`	vraie	non vérifiée
V3	`F (u1 != attente & u2 != attente)`	vraie	vraie

> **Toutes les propriétés sont vraies sur le modèle correct** (12/12). Le modèle incorrect viole S1 (borne des places) et S3 (double réservation).

Analyse du contre-exemple

Pour S1 comme pour S3, NuSMV produit la même trace de cinq états (sortie complète : `4-nusmv/resultat_bug.txt`) :

ÉTAT	U1	U2	LIBRES	CE QUI SE PASSE
1.1	attente	attente	1	État initial, une place libre
1.2	verifiee	attente	1	U1 constate qu'une place est libre
1.3	verifiee	verifiee	1	U2 constate aussi la place libre : personne n'a encore décrémenté
1.4	verifiee	confirmee	0	U2 prend la place
1.5	confirmee	confirmee	**−1**	U1 confirme sur la foi d'une vérification périmée : **double réservation**

> **Pourquoi S2 reste vraie.** L'égalité `libres + nb_confirmees = N` tient même dans le modèle incorrect, car chaque confirmation décrémente le compteur : à l'état 1.5, `−1 + 2 = 1`. Un invariant de comptage seul ne détecte donc pas la sur-réservation ; il faut aussi la borne `libres ≥ 0` (S1) ou l'exclusion directe (S3).

Correction. Le défaut vient de l'intervalle entre la lecture de `libres` et sa modification. Rendre ces deux opérations atomiques (verrou, section critique ou opération *compare-and-set*) supprime l'entrelacement fautif : c'est exactement le modèle `reservation_correcte.smv`, sur lequel toutes les propriétés sont vraies.

SECTION 8

Modèle probabiliste avec PRISM

Le modèle `5-prism/reservation.pm` est une chaîne de Markov à temps discret. À chaque pas, la demande en attente est soit traitée, soit retardée, soit perdue à cause d'une panne ; une demande perdue doit être renvoyée, ce qui coûte un pas. Pendant ce temps, d'autres clients peuvent prendre les places restantes. Si plus aucune place n'est libre, la demande est refusée.

PARAMÈTRE	VALEUR	SIGNIFICATION
`N`	2	places libres au départ
`p_retard`	0,2	la demande est retardée d'un pas
`p_echec`	0,1	panne : la demande est perdue et renvoyée
`p_autre`	0,3	un autre client prend une place pendant le pas

```
prism 5-prism/reservation.pm 5-prism/proprietes.pctl -const T=1:10
```

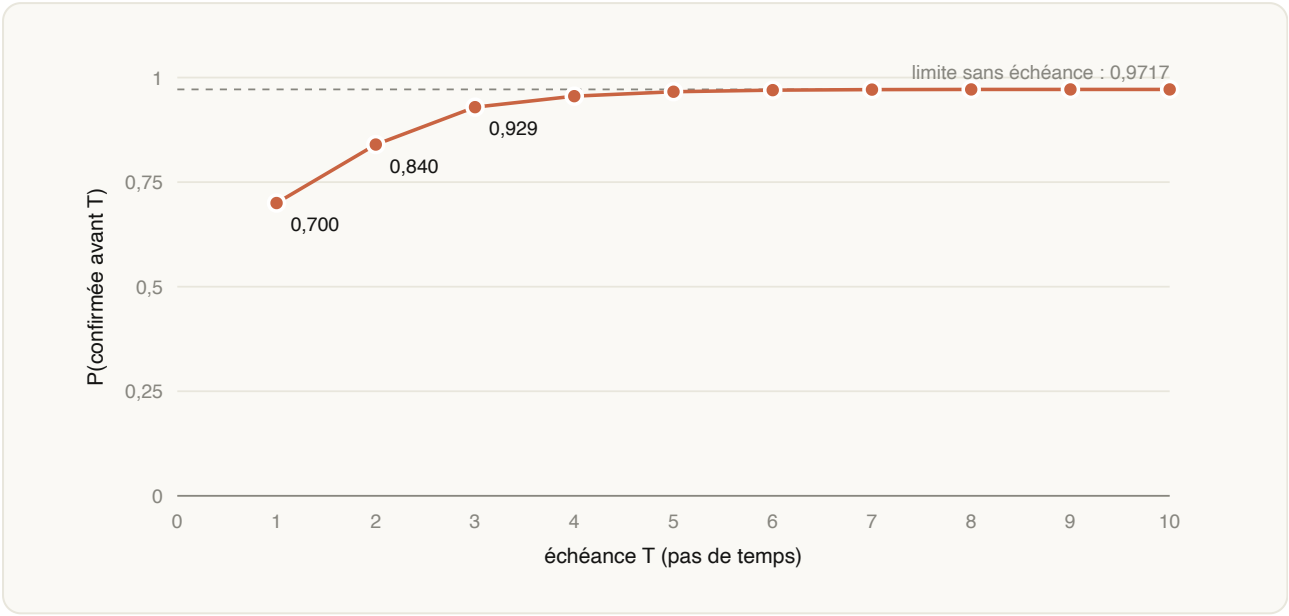


Figure 5. Probabilité que la réservation soit confirmée avant l'échéance `T`, `P=? [ F<=T "confirmee" ]` (PRISM 4.11).

T	1	2	3	4	5	6	7	8	9	10
P	0,700	0,840	0,929	0,956	0,966	0,970	0,971	0,971	0,972	0,972

PROPRIÉTÉ PCTL	RÉSULTAT
`P=? [ F "confirmee" ]`	0,9717
`P=? [ F "refusee" ]`	0,0283
`P>=1 [ F ("confirmee" | "refusee") ]`	**vraie** : toute demande est traitée

Interprétation. Sans panne ni retard, la demande serait confirmée au premier pas. Les défaillances la retardent : 70 % de confirmations au premier pas, 92,9 % avant trois pas. La courbe se stabilise vers 0,9717 à partir de `T = 6`. Les pannes et les retards ne provoquent un refus (2,8 %) que si les autres clients prennent toutes les places pendant l'attente.

SECTION 9

Conclusion

- Le contrat de `reserver` (préconditions, postconditions, invariant) est respecté par l'implémentation séquentielle : 10/10 tests réussis.
- En concurrence, une réservation non atomique permet la double réservation de la dernière place. NuSMV le démontre par un contre-exemple de cinq états.
- La version atomique vérifie toutes les propriétés de sûreté et, sous équité, de vivacité. Le réseau de Petri confirme ce résultat : 1-borné, invariant de places respecté, aucune double réservation.
- PRISM quantifie l'effet des pannes et des retards : 97,2 % des demandes sont confirmées, dont 92,9 % avant trois unités de temps.

La vérification formelle a trouvé un défaut que des tests séquentiels ne pouvaient pas révéler, et elle a prouvé l'absence de ce défaut dans la version corrigée sur l'ensemble des exécutions possibles.

Fichiers livrés

FICHIER	CONTENU
`1-java/src/*.java`	Implémentation séquentielle avec contrats, scénarios de test (`Main`)
`2-etats/etats.puml`, `etats.jff`	Diagramme d'états PlantUML, automate JFLAP
`2-etats/concurrence.puml`	Scénario des deux utilisateurs (versions incorrecte et correcte)
`3-petri/reservation.pnml`	Réseau de Petri (PIPE)
`4-nusmv/reservation_correcte.smv`, `reservation_bug.smv`	Modèles NuSMV et propriétés CTL/LTL
`4-nusmv/resultat_*.txt`	Sorties NuSMV, dont le contre-exemple
`5-prism/reservation.pm`, `proprietes.pctl`	Modèle PRISM et propriétés PCTL
`5-prism/resultats.txt`	Résultats PRISM pour `T = 1..10`

ANNEXE A

Sources

A.1 · 1-java/src/SystemeReservation.java

```
/**
 * Systeme de reservation sequentiel pour un evenement de capacite fixe.
 *
 * Invariant :
 *   0 <= placesLibres <= capacite
 *   placesLibres + nbConfirmees == capacite   (aucune place perdue ni attribuee deux fois)
 *
 * Les preconditions, postconditions et l'invariant sont verifies a chaque appel.
 * Une violation leve une IllegalStateException.
 */
public class SystemeReservation {
    private final int capacite;
    private int placesLibres;
    private int nbConfirmees = 0;

    public SystemeReservation(int capacite) {
        exiger(capacite > 0, "la capacite doit etre positive");
        this.capacite = capacite;
        this.placesLibres = capacite;
        verifierInvariant();
    }

    /**
     * Pre  : d != null et d est EN_ATTENTE.
     * Post : si une place etait libre, d est CONFIRMEE et placesLibres a diminue de 1,
     *        sinon d est REFUSEE et placesLibres n'a pas change.
     */
    public void reserver(Demande d) {
        // Preconditions
        exiger(d != null, "pre : la demande ne doit pas etre nulle");
        exiger(d.getEtat() == Etat.EN_ATTENTE, "pre : la demande doit etre EN_ATTENTE");
        verifierInvariant();
        int avant = placesLibres;

        if (placesLibres > 0) {
            placesLibres--;
            nbConfirmees++;
            d.setEtat(Etat.CONFIRMEE);
        } else {
            d.setEtat(Etat.REFUSEE);
        }

        // Postconditions
        if (avant > 0) {
            exiger(d.getEtat() == Etat.CONFIRMEE && placesLibres == avant - 1,
                   "post : une place etait libre, la demande doit etre confirmee");
        } else {
            exiger(d.getEtat() == Etat.REFUSEE && placesLibres == avant,
                   "post : aucune place libre, la demande doit etre refusee");
        }
        verifierInvariant();
    }

    /**
     * Pre  : d est EN_ATTENTE ou CONFIRMEE.
     * Post : d est ANNULEE. Si elle etait confirmee, sa place est liberee.
     */
    public void annuler(Demande d) {
        exiger(d != null, "pre : la demande ne doit pas etre nulle");
        exiger(d.getEtat() == Etat.EN_ATTENTE || d.getEtat() == Etat.CONFIRMEE,
```

```
                "pre : seule une demande en attente ou confirmee peut etre annulee");
        verifierInvariant();
        int avant = placesLibres;
        boolean etaitConfirmee = d.getEtat() == Etat.CONFIRMEE;

        if (etaitConfirmee) {
            placesLibres++;
            nbConfirmees--;
        }
        d.setEtat(Etat.ANNULEE);

        exiger(d.getEtat() == Etat.ANNULEE, "post : la demande doit etre annulee");
        exiger(placesLibres == (etaitConfirmee ? avant + 1 : avant),
                "post : la place doit etre liberee si la demande etait confirmee");
        verifierInvariant();
    }

    public int getPlacesLibres() { return placesLibres; }
    public int getCapacite() { return capacite; }

    private void verifierInvariant() {
        exiger(placesLibres >= 0 && placesLibres <= capacite,
                "invariant : 0 <= placesLibres <= capacite");
        exiger(placesLibres + nbConfirmees == capacite,
                "invariant : placesLibres + nbConfirmees == capacite");
    }

    private static void exiger(boolean condition, String message) {
        if (!condition) throw new IllegalStateException(message);
    }
}
```

A.2 · 4-nusmv/reservation_correcte.smv

```
-- Reservation de la derniere place par deux utilisateurs : VERSION CORRECTE.
-- La verification "place libre ?" et la decrementation se font en une seule
-- etape atomique (comme sous un verrou).
-- Execution : NuSMV reservation_correcte.smv

MODULE main
VAR
  u1     : {attente, confirmee, refusee, annulee};
  u2     : {attente, confirmee, refusee, annulee};
  libres : -1..1;        -- capacite N = 1. La valeur -1 signalerait une sur-reservation.
  qui    : {1, 2};       -- ordonnanceur : l'utilisateur qui agit a ce pas

DEFINE
  N := 1;
  nb_confirmees := toint(u1 = confirmee) + toint(u2 = confirmee);

ASSIGN
  init(u1)     := attente;
  init(u2)     := attente;
  init(libres) := N;

  next(u1) := case
    qui = 1 & u1 = attente & libres > 0 : {confirmee, annulee};
    qui = 1 & u1 = attente              : {refusee, annulee};
    qui = 1 & u1 = confirmee            : {confirmee, annulee};  -- peut annuler
    TRUE                                : u1;
  esac;

  next(u2) := case
    qui = 2 & u2 = attente & libres > 0 : {confirmee, annulee};
    qui = 2 & u2 = attente              : {refusee, annulee};
```

```
    qui = 2 & u2 = confirmee           : {confirmee, annulee};
    TRUE                               : u2;
  esac;

  -- Les gardes "libres > 0" et "libres < N" restent toujours vraies dans les etats
  -- accessibles. Elles servent seulement a NuSMV pour verifier le domaine de libres.
  next(libres) := case
    u1 = attente   & next(u1) = confirmee & libres > 0 : libres - 1;   -- reservation atomique
    u2 = attente   & next(u2) = confirmee & libres > 0 : libres - 1;
    u1 = confirmee & next(u1) = annulee   & libres < N : libres + 1;   -- annulation : place liberee
    u2 = confirmee & next(u2) = annulee   & libres < N : libres + 1;
    TRUE                                               : libres;
  esac;

-- Ordonnanceur equitable : chaque utilisateur agit infiniment souvent.
FAIRNESS qui = 1;
FAIRNESS qui = 2;

-- ===== Proprietes de surete (rien de mauvais n'arrive) =====
-- S1 : le nombre de places libres reste dans [0, N]
SPEC AG (libres >= 0 & libres <= N)
-- S2 : coherence, places libres + places confirmees = capacite
SPEC AG (libres + nb_confirmees = N)
-- S3 : pas de double reservation de la derniere place
SPEC AG !(u1 = confirmee & u2 = confirmee)
-- S4 : une demande refusee ou annulee le reste (etats finaux)
SPEC AG (u1 = refusee -> AG u1 = refusee)
SPEC AG (u1 = annulee -> AG u1 = annulee)
-- S5 : une demande n'est refusee que s'il n'y a plus de place
SPEC AG ((u1 = attente & libres > 0) -> AX u1 != refusee)

-- ===== Proprietes de vivacite (quelque chose de bon finit par arriver) =====
-- V1 : toute demande en attente recoit une reponse
SPEC AG (u1 = attente -> AF u1 != attente)
SPEC AG (u2 = attente -> AF u2 != attente)
-- V2 : chaque utilisateur peut obtenir la place (atteignabilite)
SPEC EF (u1 = confirmee)
SPEC EF (u2 = confirmee)
-- V3 : en LTL, les deux demandes finissent par etre traitees
LTLSPEC F (u1 != attente & u2 != attente)
-- V4 : une place liberee peut toujours etre reprise par l'autre utilisateur
SPEC AG ((u1 = annulee & u2 = attente) -> EF u2 = confirmee)
```

A.3 · 4-nusmv/reservation_bug.smv

```
-- Reservation de la derniere place par deux utilisateurs : VERSION INCORRECTE.
-- La reservation se fait en deux etapes non atomiques :
--   1. verifier : l'utilisateur lit "libres > 0" et passe dans l'etat interne 'verifiee'
--   2. confirmer : il decremente libres SANS reverifier
-- Entre les deux, l'autre utilisateur peut aussi verifier : les deux obtiennent la place.
-- Execution : NuSMV reservation_bug.smv  (les proprietes de surete S1 et S3 sont FAUSSES)

MODULE main
VAR
  u1     : {attente, verifiee, confirmee, refusee, annulee};
  u2     : {attente, verifiee, confirmee, refusee, annulee};
  libres : -1..1;        -- capacite N = 1. La valeur -1 signale une sur-reservation.
  qui    : {1, 2};

DEFINE
  N := 1;
  nb_confirmees := toint(u1 = confirmee) + toint(u2 = confirmee);

ASSIGN
  init(u1)     := attente;
```

```
  init(u2)     := attente;
  init(libres) := N;

  next(u1) := case
    qui = 1 & u1 = attente & libres > 0 : {verifiee, annulee};
    qui = 1 & u1 = attente              : {refusee, annulee};
    qui = 1 & u1 = verifiee             : confirmee;              -- pas de nouvelle verification
    qui = 1 & u1 = confirmee            : {confirmee, annulee};
    TRUE                                : u1;
  esac;

  next(u2) := case
    qui = 2 & u2 = attente & libres > 0 : {verifiee, annulee};
    qui = 2 & u2 = attente              : {refusee, annulee};
    qui = 2 & u2 = verifiee             : confirmee;
    qui = 2 & u2 = confirmee            : {confirmee, annulee};
    TRUE                                : u2;
  esac;

  -- Les gardes sur libres ne servent qu'a rester dans le domaine -1..1.
  -- "libres >= 0" n'est PAS une reverification : elle laisse passer libres = 0, d'ou le -1.
  next(libres) := case
    u1 = verifiee  & next(u1) = confirmee & libres >= 0 : libres - 1;   -- decrementation tardive
    u2 = verifiee  & next(u2) = confirmee & libres >= 0 : libres - 1;
    u1 = confirmee & next(u1) = annulee   & libres < N  : libres + 1;
    u2 = confirmee & next(u2) = annulee   & libres < N  : libres + 1;
    TRUE                                                : libres;
  esac;

FAIRNESS qui = 1;
FAIRNESS qui = 2;

-- Memes proprietes que la version correcte
SPEC AG (libres >= 0 & libres <= N)               -- S1 : FAUSSE
SPEC AG (libres + nb_confirmees = N)              -- S2
SPEC AG !(u1 = confirmee & u2 = confirmee)        -- S3 : FAUSSE (double reservation)
SPEC AG (u1 = attente -> AF u1 != attente)        -- V1
LTLSPEC F (u1 != attente & u2 != attente)         -- V3
```

A.4 · 5-prism/reservation.pm

```
// Reservation avec defaillances et retards (chaine de Markov a temps discret).
// Un pas = une unite de temps.
// A chaque pas, la demande en attente est soit traitee, soit retardee, soit perdue
// (panne : elle doit etre renvoyee, ce qui coute un pas).
// Pendant ce temps, d'autres clients peuvent prendre les places restantes.

dtmc

const int N = 2;              // places libres au depart
const double p_retard = 0.2;  // la demande est retardee d'un pas
const double p_echec  = 0.1;  // panne : la demande est perdue et doit etre renvoyee
const double p_autre  = 0.3;  // un autre client prend une place pendant ce pas

formula p_ok = 1 - p_retard - p_echec;   // la demande est traitee a ce pas

// Etats de la demande
const int ATTENTE   = 0;
const int CONFIRMEE = 1;
const int REFUSEE   = 2;
const int PANNE     = 3;   // demande perdue, renvoi au pas suivant

module reservation
  etat   : [0..3] init ATTENTE;
  libres : [0..N] init N;
```

```
  // Demande traitee, retardee ou perdue. Un autre client peut prendre une place.
  [] etat = ATTENTE & libres > 0 ->
        p_ok                : (etat' = CONFIRMEE) & (libres' = libres - 1)
      + p_retard * (1 - p_autre) : true
      + p_retard * p_autre       : (libres' = libres - 1)
      + p_echec  * (1 - p_autre) : (etat' = PANNE)
      + p_echec  * p_autre       : (etat' = PANNE) & (libres' = libres - 1);

  // Plus de place : la demande est refusee.
  [] etat = ATTENTE & libres = 0 -> (etat' = REFUSEE);

  // Renvoi apres une panne (un autre client peut encore prendre une place).
  [] etat = PANNE & libres > 0 ->
        (1 - p_autre) : (etat' = ATTENTE)
      + p_autre       : (etat' = ATTENTE) & (libres' = libres - 1);
  [] etat = PANNE & libres = 0 -> (etat' = ATTENTE);

  // Etats finaux
  [] etat = CONFIRMEE | etat = REFUSEE -> true;
endmodule

label "confirmee" = etat = CONFIRMEE;
label "refusee"   = etat = REFUSEE;
```

A.5 · 5-prism/proprietes.pctl

```
// T : echeance en nombre de pas (passee avec -const T=...)
const int T;

// P1 : probabilite que la reservation soit confirmee avant l'echeance T
P=? [ F<=T "confirmee" ]

// P2 : probabilite que la reservation soit finalement confirmee (sans echeance)
P=? [ F "confirmee" ]

// P3 : probabilite que la reservation soit refusee
P=? [ F "refusee" ]

// P4 : la demande est traitee (confirmee ou refusee) avec probabilite 1
P>=1 [ F ("confirmee" | "refusee") ]
```